

Using Basic AI to Automate FreeCAD

OverView

There are currently several different ways to utilize AI to help facilitate the automation of FreeCAD. However, the method I would like to discuss has the lowest barriers of entry and incurs no additional costs.

AI Chat-Bots

The three AI chatbot interfaces I tested with this methodology were Microsoft Copilot, Google Gemini, and OpenAI's ChatGPT. Based on my results as of October 2026, the free versions of Gemini and ChatGPT both performed quite well. Unfortunately, Microsoft Copilot struggled to complete my requests successfully.

I suspect that Anthropic's Claude would also perform well at these tasks. However, because using Claude's chatbot can involve costs, I excluded it from my testing

Belt Return

The example I will use to illustrate my methodology is the creation of a belt return.

For some odd reason, most of my belts have a tendency to flop around rather distressingly. This is likely because the belt loops on my pants are in the wrong places, or because the belts' built-in returns are not positioned where I need them. My usual solution is to create my own supplemental belt returns.

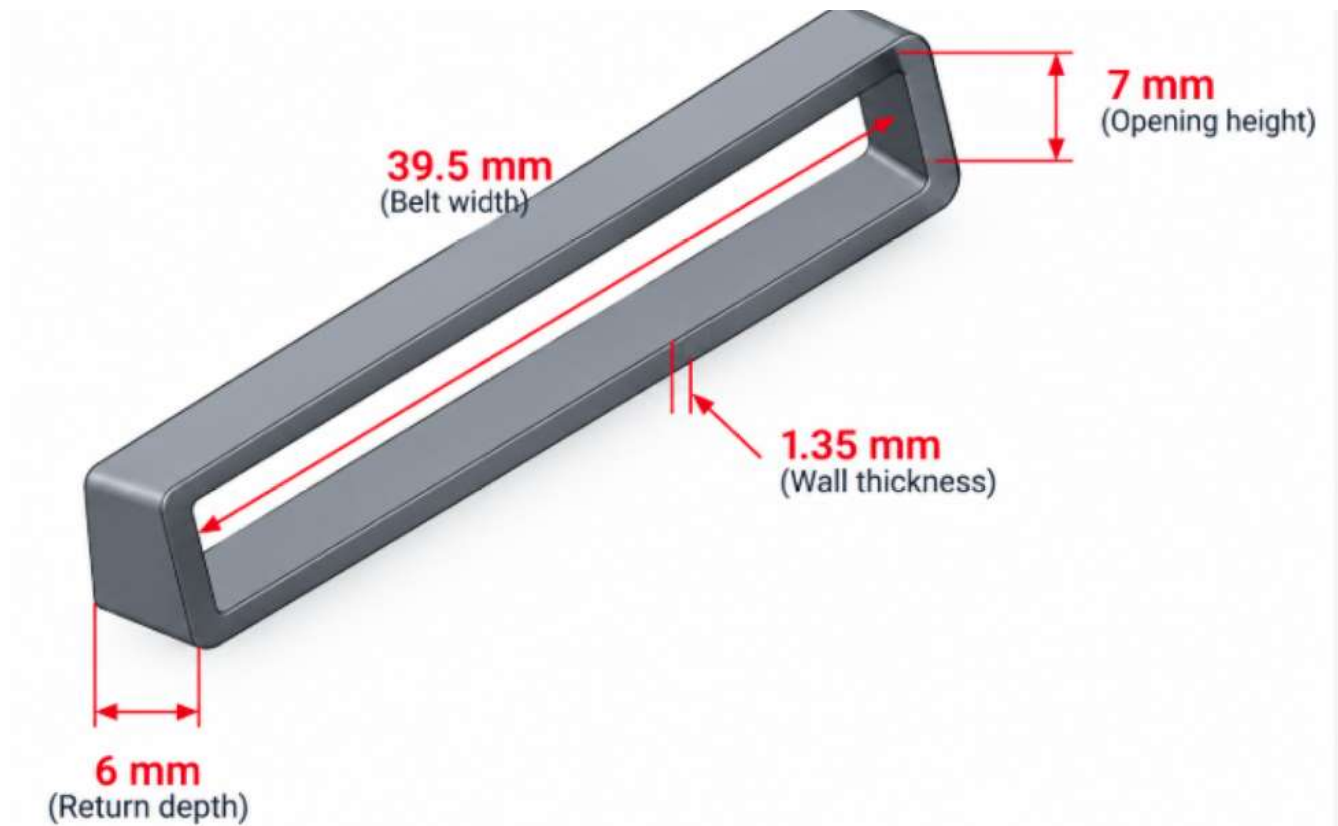
Since I own a 3D printer and have the basic skills needed to design a suitable model in FreeCAD, creating one myself would probably take an hour or two. However, either because I am exceedingly lazy or because I want to explore the capabilities of AI, I have decided to use an AI chatbot to speed up the process. I also hope to improve the functionality of my FreeCAD models so that they can more easily accommodate different belt sizes.

The process begins by measuring a few key dimensions of your existing belt.

The first measurement is the thickness of the belt. For my design, I chose an opening height equal to twice the belt thickness, allowing room for two layers of belt. Since my belt is 3.5 mm thick, the opening height is 7 mm.

The next measurement is the width of the belt, which in my case is 39.5 mm.

Finally, two additional dimensions need to be selected: the depth of the belt return and the thickness of its walls. For my design, I chose a depth of 6 mm and a wall thickness of 1.35 mm.



In summary, here are the key parameters for my belt return.

Parameter Description	Dimension
Belt Thickness	3.5 mm
Belt Retainer Opening Width	7 mm (Belt Thickness * 2)
Belt Thickness Height	39.5 mm
Belt Retainer Width	6 mm
Belt Retainer Wall Thickness	1.35mm

Following the clear definition of your key starting parameters, it is time to start your conversation with your favorite AI chatbot.

Here is a sample of my conversation:

User: Do you know how to create FreeCAD macros?

User: I want you to create a FreeCAD macro that creates a belt return for my belt. The key parameters are as follows: Belt Thickness = 3.5 mm, Belt Retainer Opening Width = 2 x Belt Thickness, Belt Thickness Height = 39.5 mm, Belt Retainer Width = 6 mm, and Belt Retainer Wall Thickness = 1.35 mm.

After providing this information to the AI, I suggest confirming that it fully understands the objective and can explain the significance of each specified parameter.

Once the first iteration is completed successfully, you can later provide the AI with additional instructions to modify the macro. For example, you can ask it to generate a built-in spreadsheet to facilitate making future fine adjustments to the belt's dimensions.

In addition, I would suggest asking the AI to create a prompt to fully document the request.

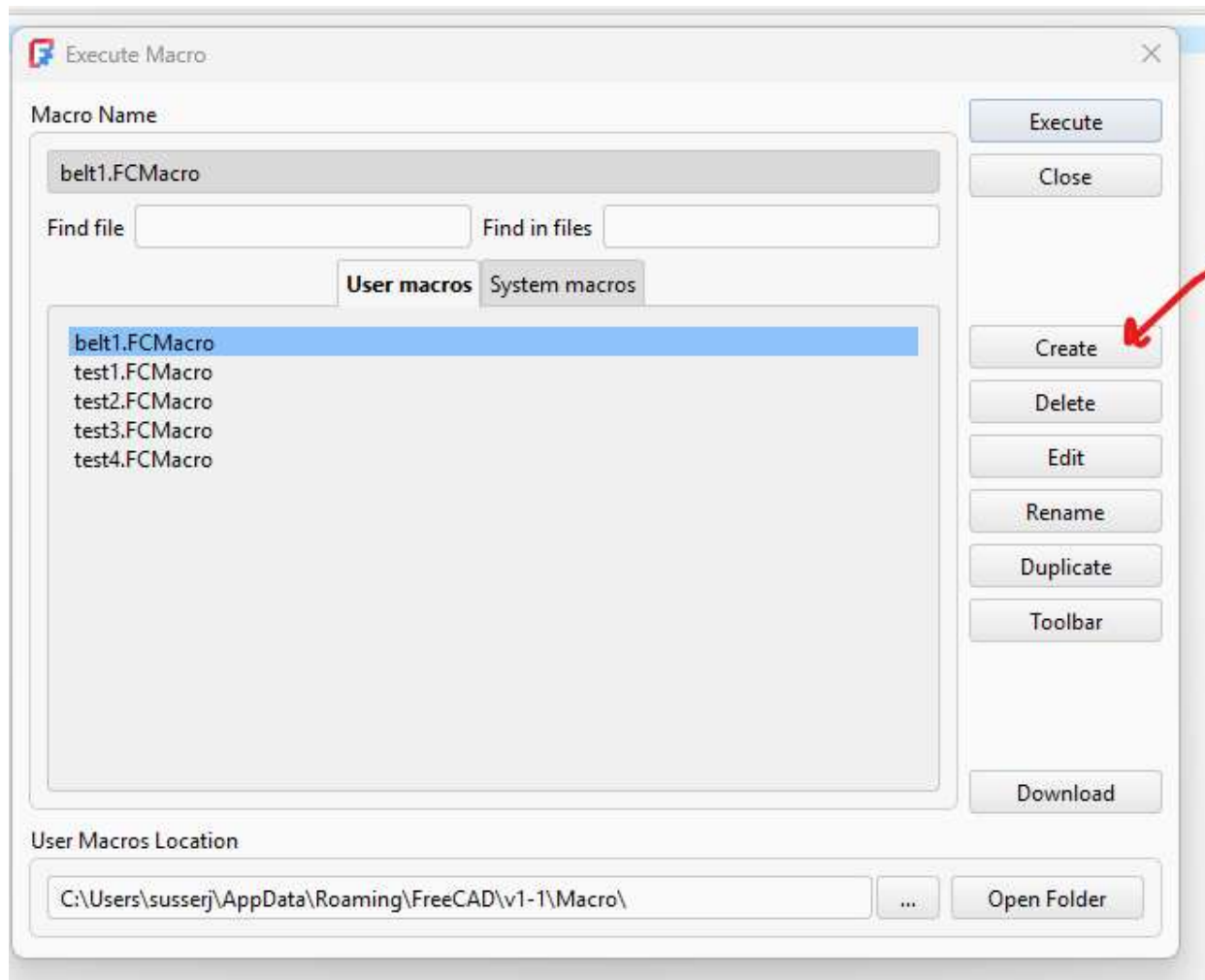
The specific assets for this project will be posted in my GitHub repository:

- **AI Prompt**
- **BeltKeeperPrompt.FCStd** (FreeCAD Project File)
- **BeltKeeper.FCMacro** (FreeCAD Macro)
- **BeltKeeper.stl** (3D Printable File)

Running the FreeCAD Macro

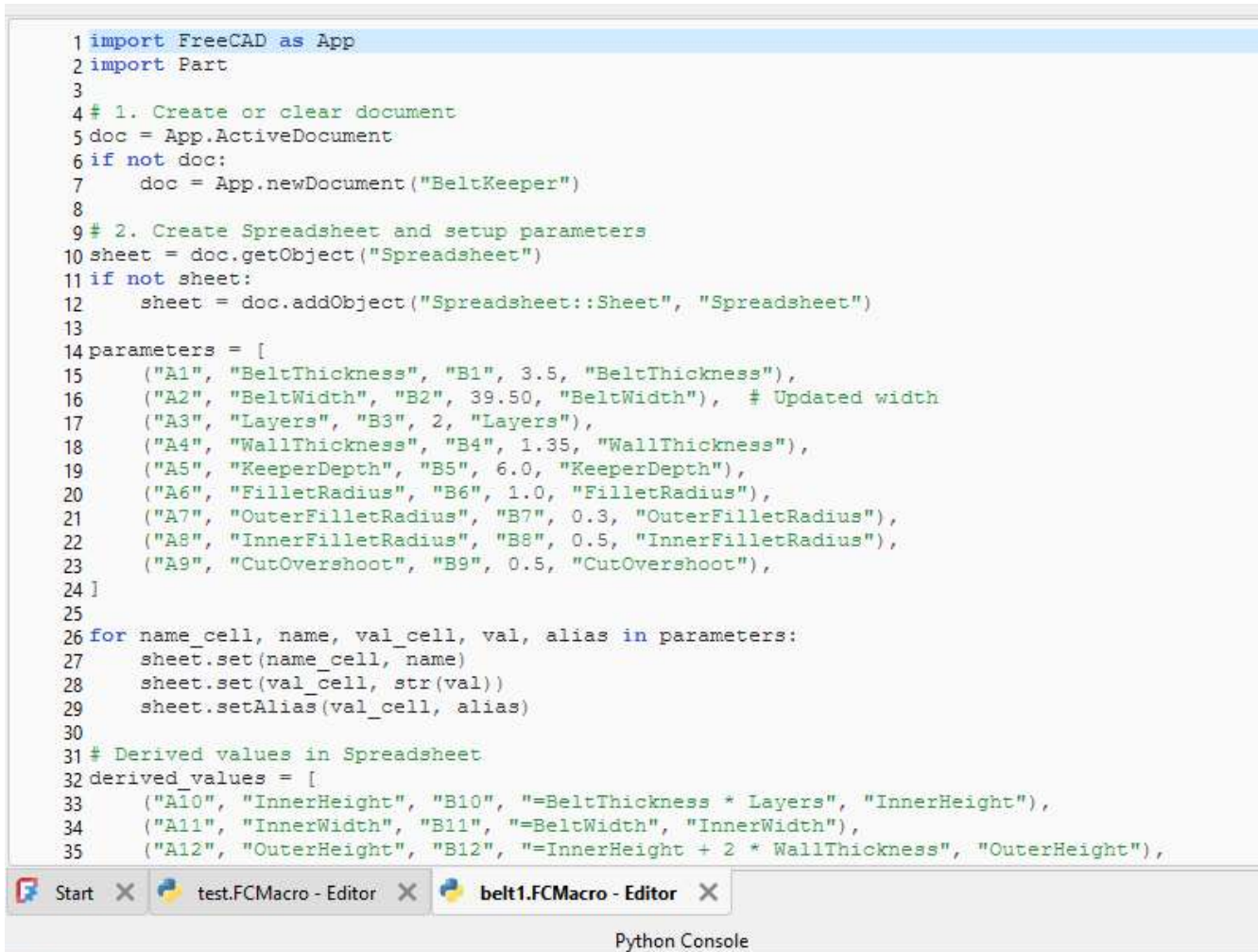
Running the provided FreeCAD macro is straightforward:

1. **Create a new document** (File > New or Ctrl+N).
2. **Open the Macro dialog** by selecting **Macro > Macros...** from the top menu bar.
3. **Click the Create button** in the Execute Macro dialog window

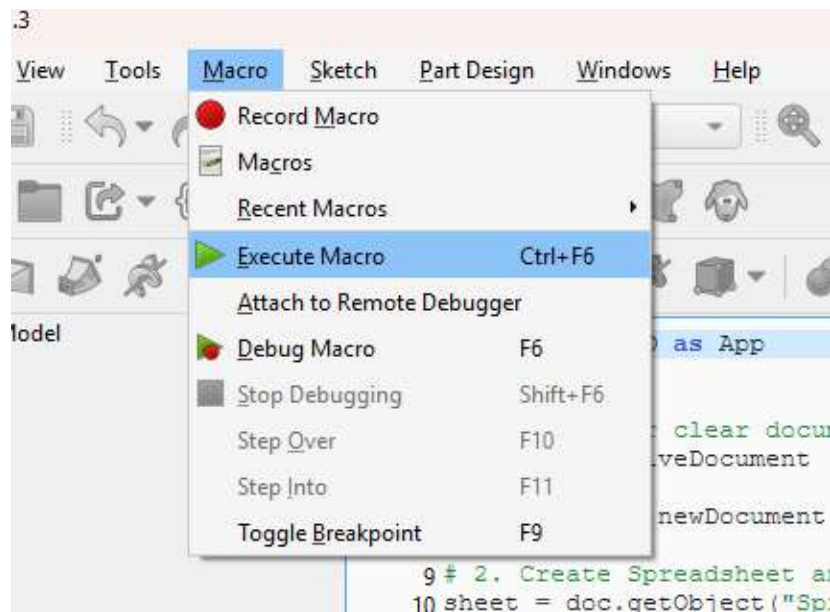


4. Provide a unique name for the macro (e.g., `BeltKeeper.FCMacro` or `belt1.FCMacro`) when prompted and click OK.
5. Paste the code: Paste the macro script generated by your AI directly into the FreeCAD editor tab
6. Save and execute: Save the macro (`Ctrl+S`), then click Execute (or press `F6`) in the macro toolbar to create the model and project spreadsheet

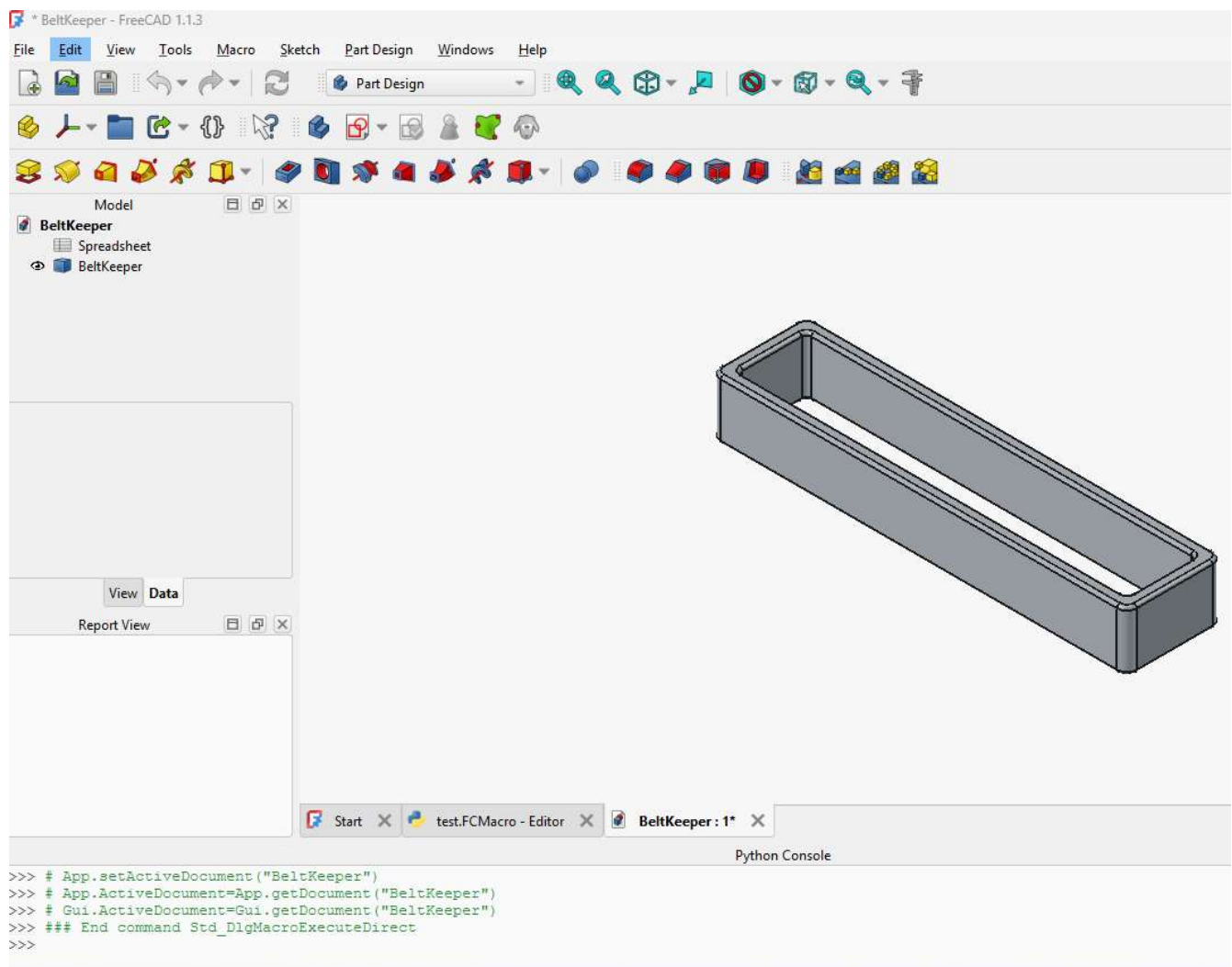
```
1 import FreeCAD as App
2 import Part
3
4 # 1. Create or clear document
5 doc = App.ActiveDocument
6 if not doc:
7     doc = App.newDocument("BeltKeeper")
8
9 # 2. Create Spreadsheet and setup parameters
10 sheet = doc.getObject("Spreadsheet")
11 if not sheet:
12     sheet = doc.addObject("Spreadsheet::Sheet", "Spreadsheet")
13
14 parameters = [
15     ("A1", "BeltThickness", "B1", 3.5, "BeltThickness"),
16     ("A2", "BeltWidth", "B2", 39.50, "BeltWidth"), # Updated width
17     ("A3", "Layers", "B3", 2, "Layers"),
18     ("A4", "WallThickness", "B4", 1.35, "WallThickness"),
19     ("A5", "KeeperDepth", "B5", 6.0, "KeeperDepth"),
20     ("A6", "FilletRadius", "B6", 1.0, "FilletRadius"),
21     ("A7", "OuterFilletRadius", "B7", 0.3, "OuterFilletRadius"),
22     ("A8", "InnerFilletRadius", "B8", 0.5, "InnerFilletRadius"),
23     ("A9", "CutOvershoot", "B9", 0.5, "CutOvershoot"),
24 ]
25
26 for name_cell, name, val_cell, val, alias in parameters:
27     sheet.set(name_cell, name)
28     sheet.set(val_cell, str(val))
29     sheet.setAlias(val_cell, alias)
30
31 # Derived values in Spreadsheet
32 derived_values = [
33     ("A10", "InnerHeight", "B10", "=BeltThickness * Layers", "InnerHeight"),
34     ("A11", "InnerWidth", "B11", "=BeltWidth", "InnerWidth"),
35     ("A12", "OuterHeight", "B12", "=InnerHeight + 2 * WallThickness", "OuterHeight"),
```



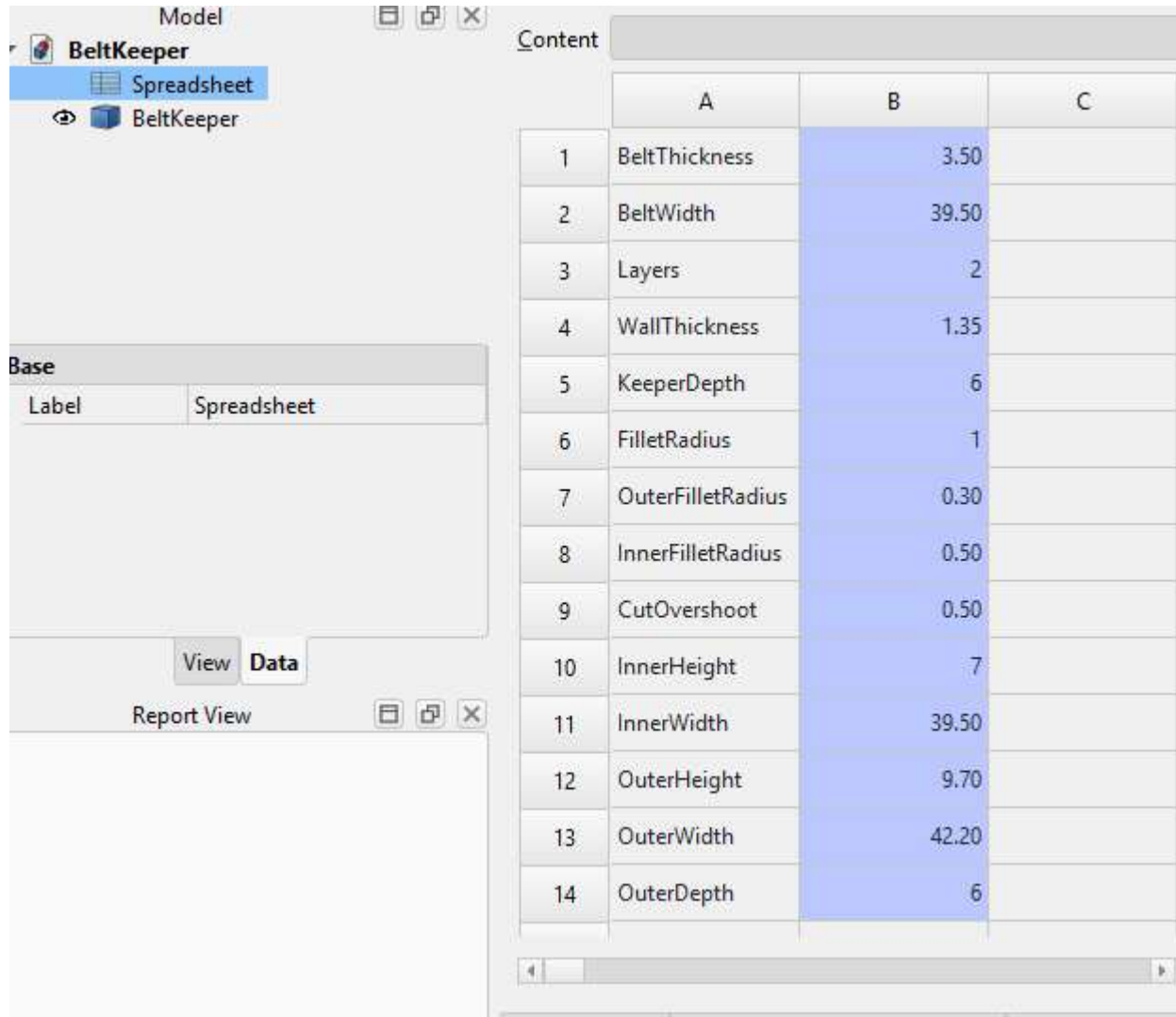
7. And finally, execute the macro and observe the results produced by the execution of the macro.



If all goes well, you should see a result screen in FreeCAD similar to the following:



Key values can be further adjusted from the FreeCAD Spreadsheet Object



	A	B	C
1	BeltThickness	3.50	
2	BeltWidth	39.50	
3	Layers	2	
4	WallThickness	1.35	
5	KeeperDepth	6	
6	FilletRadius	1	
7	OuterFilletRadius	0.30	
8	InnerFilletRadius	0.50	
9	CutOvershoot	0.50	
10	InnerHeight	7	
11	InnerWidth	39.50	
12	OuterHeight	9.70	
13	OuterWidth	42.20	
14	OuterDepth	6	

Troubleshooting: What Happens If Something Goes Wrong?

It is unlikely that this method will work perfectly on the very first try without any issues whatsoever. Fortunately, troubleshooting and correcting problems is usually straightforward.

You have four key steps and tools for providing feedback to the AI when issues arise:

- 1. Describe the issue clearly:** Explain your analysis of the generated object. Detail exactly what you see versus what you expected to see as clearly as possible.
- 2. Provide a screenshot:** Take a screen capture of the model rendered in FreeCAD and share it with the AI.
- 3. Share Python Console output:** Copy and paste the text from FreeCAD's Python Console. This is especially critical if the console displays error messages.

4. **Request the fix:** Ask the AI if it understands what went wrong and request an updated macro script to fix the problem.

Repeat these steps iteratively until the object renders correctly as required and the spreadsheet parameters can be modified to apply any necessary fine adjustments.

Printing the Object

Printing the 3D-generated object created in FreeCAD is straightforward:

1. **Export the 3D model:** Select the object in FreeCAD, then go to **File > Export**. Select **STL Mesh (*.stl .ast)* as the file format.
2. **Slice the model:** Import the exported STL file into your slicer software (e.g., PrusaSlicer, Cura, or Bambu Studio) and slice it to generate the **G-code** file.
3. **Send to printer:** Transfer the G-code file to your 3D printer.

Don't be surprised if your first print is slightly off in terms of the required dimensions—a few test iterations are to be expected to dial in the fit.

Post-Processing

Following a successful print, a light sanding may be required using 180-grit sandpaper. Given that my belt returns are printed in black PLA, I have found that a few strokes of a black Sharpie marker fully hide any remaining sanding marks.



Appendix A (Beltkeeper AI Prompt)

Macro Generation Prompt

Task: Write a FreeCAD Python macro that creates a parametric 3D print model for a **Belt Keeper** using a **FreeCAD Spreadsheet Object** for parameter management.

Requirements:

1. Parametric Setup (**FeaturePython**):

- Create a `Spreadsheet::Sheet` named "Spreadsheet" containing all driving parameters and derived expressions.
- Build the geometry inside a `Part::FeaturePython` object ("BeltKeeper") with an `execute(self, obj)` method so that whenever values in the Spreadsheet are updated, the 3D model automatically recomputes and adjusts its dimensions.
- Add a property link connecting `BeltKeeper.Spreadsheet = sheet` to enforce dynamic recomputation dependencies.

2. Spreadsheet Parameters to Include:

- **User Inputs (Values & Aliases):**
 - `BeltThickness: 3.5`
 - `BeltWidth: 39.50`
 - `Layers: 2`
 - `WallThickness: 1.35`
 - `KeeperDepth: 6.0`
 - `FilletRadius: 1.0` (Outer vertical corner fillets)
 - `OuterFilletRadius: 0.3` (Entry rim roundovers)
 - `InnerFilletRadius: 0.5` (Inner cavity corner fillets)
 - `CutOvershoot: 0.5` (Extra length for clean Boolean subtraction)
- **Derived Formula Cell Aliases:**
 - `InnerHeight: =BeltThickness * Layers`
 - `InnerWidth: =BeltWidth`
 - `OuterHeight: =InnerHeight + 2 * WallThickness`
 - `OuterWidth: =InnerWidth + 2 * WallThickness`

- `OuterDepth: =KeeperDepth`

3. Geometry Construction Logic (**execute method**):

- Cast all values fetched from `sheet.get()` to `float()` to avoid type/unit mismatch errors.
- **Outer Block:** Centered at $X=0, Y=0$, starting at $Z=0$, sized to $\text{OuterWidth} \times \text{OuterHeight} \times \text{OuterDepth}$.
- **Inner Cutout:** Centered at $X=0, Y=0$, extending from $Z=-\text{CutOvershoot}$ to $Z=\text{OuterDepth} + \text{CutOvershoot}$, sized to $\text{InnerWidth} \times \text{InnerHeight} \times (\text{OuterDepth} + 2 * \text{CutOvershoot})$.
- **Fillets:**
 - Filter and apply `FilletRadius` to vertical outer corner edges.
 - Filter and apply `InnerFilletRadius` to vertical inner cavity edges.
 - Filter and apply `OuterFilletRadius` to front/back entry rim edges (at $Z=0$ and $Z=\text{OuterDepth}$).

4. GUI/View Setup:

- Set active view to axonometric/isometric and execute `ViewFit` after script completion.

Appendix B (BeltKeeper.FCMacro)

import FreeCAD as App import Part

1. Create or clear document

```
doc = App.ActiveDocument if not doc: doc = App.newDocument("BeltKeeper")
```

2. Create Spreadsheet and setup parameters

```
sheet = doc.getObject("Spreadsheet") if not sheet: sheet = doc.addObject("Spreadsheet::Sheet", "Spreadsheet")
```

```
parameters = [ ("A1", "BeltThickness", "B1", 3.5, "BeltThickness"), ("A2", "BeltWidth", "B2", 39.50, "BeltWidth"), # Updated width ("A3", "Layers", "B3", 2, "Layers"), ("A4", "WallThickness", "B4", 1.35, "WallThickness"), ("A5", "KeeperDepth", "B5", 6.0, "KeeperDepth"), ("A6", "FilletRadius", "B6", 1.0, "FilletRadius"), ("A7", "OuterFilletRadius", "B7", 0.3, "OuterFilletRadius"), ("A8", "InnerFilletRadius", "B8", 0.5, "InnerFilletRadius"), ("A9", "CutOvershoot", "B9", 0.5, "CutOvershoot"), ]
```

```
for name_cell, name, val_cell, val, alias in parameters: sheet.set(name_cell, name) sheet.set(val_cell, str(val)) sheet.setAlias(val_cell, alias)
```

Derived values in Spreadsheet

```
derived_values = [ ("A10", "InnerHeight", "B10", "=BeltThickness * Layers", "InnerHeight"), ("A11", "InnerWidth", "B11", "=BeltWidth", "InnerWidth"), ("A12", "OuterHeight", "B12", "=InnerHeight + 2 * WallThickness", "OuterHeight"), ("A13", "OuterWidth", "B13", "=InnerWidth + 2 * WallThickness", "OuterWidth"), ("A14", "OuterDepth", "B14", "=KeeperDepth", "OuterDepth"), ]
```

```
for name_cell, name, val_cell, expr, alias in derived_values: sheet.set(name_cell, name) sheet.set(val_cell, expr) sheet.setAlias(val_cell, alias)
```

3. Define the Parametric Object Class

```
class ParametricBeltKeeper: def init(self, obj): obj.Proxy = self
```

```
def execute(self, obj):  
    # Retrieve the document spreadsheet  
    sheet = obj.Document.getObject("Spreadsheet")  
    if not sheet:  
        return  
  
    # Fetch computed values from spreadsheet and safely cast to float  
    outer_w = float(sheet.get("OuterWidth"))  
    outer_h = float(sheet.get("OuterHeight"))  
    outer_d = float(sheet.get("OuterDepth"))
```

```

inner_w = float(sheet.get("InnerWidth"))
inner_h = float(sheet.get("InnerHeight"))
cut_overshoot = float(sheet.get("CutOvershoot"))
fillet_r = float(sheet.get("FilletRadius"))
outer_fillet_r = float(sheet.get("OuterFilletRadius"))
inner_fillet_r = float(sheet.get("InnerFilletRadius"))

# Build Geometry
outer_box = Part.makeBox(outer_w, outer_h, outer_d)
outer_box.translate(App.Vector(-outer_w / 2.0, -outer_h / 2.0, 0.0))

cut_w = inner_w
cut_h = inner_h
cut_d = outer_d + 2.0 * cut_overshoot

inner_box = Part.makeBox(cut_w, cut_h, cut_d)
inner_box.translate(App.Vector(-cut_w / 2.0, -cut_h / 2.0, -cut_overshoot))

keeper_shape = outer_box.cut(inner_box)

# Main vertical outer corner fillets
vertical_outer_edges = []
for edge in keeper_shape.Edges:
    v_points = [v.Point for v in edge.Vertexes]
    is_vertical = (
        len(v_points) == 2
        and abs(v_points[0].x - v_points[1].x) < 1e-3
        and abs(v_points[0].y - v_points[1].y) < 1e-3
    )
    is_outer = not all(
        abs(abs(pt.x) - (inner_w / 2.0)) < 1e-3 for pt in v_points
    )

    if is_vertical and is_outer:
        vertical_outer_edges.append(edge)

if vertical_outer_edges and fillet_r > 0:
    keeper_shape = keeper_shape.makeFillet(fillet_r, vertical_outer_edges)

# Inner cavity corner fillets
vertical_inner_edges = []
for edge in keeper_shape.Edges:
    v_points = [v.Point for v in edge.Vertexes]
    is_vertical = (
        len(v_points) == 2
        and abs(v_points[0].x - v_points[1].x) < 1e-3
        and abs(v_points[0].y - v_points[1].y) < 1e-3
    )
    is_inner = all(
        abs(abs(pt.x) - (inner_w / 2.0)) < 1e-3 for pt in v_points
    )

    if is_vertical and is_inner:
        vertical_inner_edges.append(edge)

if vertical_inner_edges and inner_fillet_r > 0:
    keeper_shape = keeper_shape.makeFillet(
        inner_fillet_r, vertical_inner_edges
    )

```

```

# Entry rim fillets
rim_edges = []
for edge in keeper_shape.Edges:
    v_points = [v.Point for v in edge.Vertexes]
    is_front_or_back = all(
        abs(pt.z - 0.0) < 1e-3 or abs(pt.z - outer_d) < 1e-3
        for pt in v_points
    )
    is_outer = not all(
        abs(abs(pt.x) - (inner_w / 2.0)) < 1e-3 for pt in v_points
    )

    if is_front_or_back and is_outer:
        rim_edges.append(edge)

if rim_edges and outer_fillet_r > 0:
    keeper_shape = keeper_shape.makeFillet(outer_fillet_r, rim_edges)

# Assign updated shape to object
obj.Shape = keeper_shape

```

4. Create or Update Parametric Feature

```

keeper_obj = doc.getObject("BeltKeeper") if not keeper_obj: keeper_obj =
doc.addObject("Part::FeaturePython", "BeltKeeper") ParametricBeltKeeper(keeper_obj) if
App.GuiUp: import FreeCADGui

```

```

keeper_obj.ViewObject.Proxy = 0 # Default view provider

```

Link Spreadsheet dependency to force automatic execution on change

```

keeper_obj.addProperty( "App::PropertyLink", "Spreadsheet", "Base", "Source
spreadsheet" ) keeper_obj.Spreadsheet = sheet

doc.recompute()

```

5. Fit View

```

if App.GuiUp: import FreeCADGui

```

```

FreeCADGui.activeDocument().activeView().viewAxonometric()
FreeCADGui.SendMsgToActiveView("ViewFit")

```

Appendix C (FreeCad Spreadsheet)

1	BeltThickness	3.50
2	BeltWidth	39.50
3	Layers	2
4	WallThickness	1.35
5	KeeperDepth	6
6	FilletRadius	1
7	OuterFilletRadius	0.30
8	InnerFilletRadius	0.50
9	CutOvershoot	0.50
10	InnerHeight	7
11	InnerWidth	39.50
12	OuterHeight	9.70
13	OuterWidth	42.20
14	OuterDepth	6